🏢 Edulateral Foundation — Official Evaluative Assessment Bank

Day 5: Generative AI, Large Language Models, OpenVINO & Habana Gaudi

Target Audience: GLA University 7th Semester B.Tech Computer Science & Engineering
Lead Trainer: Shivam Gupta | **Evaluation Mode**: Objective Technical MCQ Assessment
Total Questions: 25 Questions | **Time Limit**: 45 Minutes

Question 01: D5-Q01

In Large Language Model (LLM) inference, what are the two fundamentally distinct computational phases, and what hardware resource bounds each phase?

- **(A)** Compilation phase and Linking phase.
- **(B)** Prefill Phase (Prompt processing), which is compute-bound and GEMM-heavy; and Decode Phase (Token generation), which is memory-bandwidth bound and autoregressive.
- **(C)** Training phase and Validation phase.
- **(D)** Download phase and Upload phase.

> **Correct Answer**: **(B)**
> **Technical Explanation**: During the Prefill Phase, all input prompt tokens are processed simultaneously in parallel, making it compute-bound (saturating matrix multiplication units). During the Decode Phase, tokens are generated one-by-one autoregressively; each new token requires loading the entire multi-gigabyte model weights and KV-cache from memory, making throughput bound strictly by memory bandwidth (TB/s).

Question 02: D5-Q02

Why is the memory bandwidth (TB/s) of a processor the primary limiting factor for LLM autoregressive token generation speed (Tokens/Second) during the decode phase?

- **(A)** Because token generation requires continuous internet queries.
- **(B)** Because generating each single token requires streaming the full model weights (e.g. 16 GB for FP16 8B model) from DRAM into execution registers, yielding an Arithmetic Intensity of ~1 FLOP/Byte.
- **(C)** Because CPUs cannot store strings in cache.
- **(D)** Because transformers use recursion.

> **Correct Answer**: **(B)**
> **Technical Explanation**: In autoregressive decoding with batch size 1, computing each token requires reading every parameter in the model from DRAM. For an 8-billion parameter model in 16-bit precision (16 GB), generating 1 token requires loading 16 GB of data. If memory bandwidth is 100 GB/s, maximum theoretical speed is capped at $100/16 \approx 6.25$ tokens/sec, regardless of peak TFLOPs.

Question 03: D5-Q03

What is the purpose of Key-Value (KV) Caching in transformer decoder inference?

- **(A)** It saves user passwords in browser cache.
- **(B)** It stores previously computed Key and Value attention projection tensors across past tokens in memory, avoiding redundant re-computation of past keys/values at every step of generation.
- **(C)** It compresses the vocabulary dictionary.
- **(D)** It deletes past tokens to save RAM.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Without KV caching, generating token t would require recomputing self-attention across all previous tokens 0 through $t-1$, resulting in $O(T^2)$ computational complexity per token and $O(T^3)$ total generation cost. KV caching stores past keys and values, reducing per-token computation to $O(T)$.

Question 04: D5-Q04

What is OpenVINO Toolkit and what is its primary architecture for high-performance AI deployment?

- **(A)** A Python web development framework like Django.
- **(B)** An open-source toolkit for optimizing and deploying deep learning models across Intel hardware (Xeon CPUs, Intel GPUs, NPUs), comprising Model Optimizer, runtime plugins, and graph execution optimizations.
- **(C)** A replacement for the Linux operating system.
- **(D)** A database for storing audio files.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Intel OpenVINO (Open Visual Inference and Neural Network Optimization) is a high-performance inference engine. It converts models from PyTorch, ONNX, and TensorFlow into an Intermediate Representation (IR), applies layer fusion and quantization, and executes them with target-specific hardware plugins.

Question 05: D5-Q05

What is INT4 Weight-Only Quantization (WOQ) in LLM inference optimization, and why does it drastically increase token generation throughput?

- **(A)** It converts model weights from 16-bit floats to 4-bit integers while performing activations in float, reducing memory footprint and memory bandwidth traffic by 75%.
- **(B)** It deletes 75% of the transformer layers.
- **(C)** It trains the model from scratch on 4 images.
- **(D)** It converts text tokens into 4-letter words.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Because autoregressive decoding is memory-bandwidth bound, quantizing model weights from FP16 (16 bits) to INT4 (4 bits) reduces memory bandwidth consumption by 4x. The weights are unpacked to floating point on-the-fly in registers/cache, delivering near-linear throughput increases with minimal perplexity degradation.

Question 06: D5-Q06

What package from Hugging Face integrates Intel OpenVINO directly into the `transformers` library, enabling 1-line model optimization?

- **(A)** optimum-intel[openvino]
- **(B)** scikit-learn
- **(C)** pandas-ai
- **(D)** intel-office

> **Correct Answer**: **(A)**
> **Technical Explanation**: `optimum-intel` is the extension of Hugging Face Optimum for Intel hardware. Developers can import `OVModelForCausalLM` directly instead of `AutoModelForCausalLM`, automatically utilizing OpenVINO's optimized C++ runtime engine.

Question 07: D5-Q07

What is Intel NNCF (Neural Network Compression Framework)?

- **(A)** A tool to zip files into .tar.gz archives.
- **(B)** A comprehensive Python framework for advanced post-training quantization (PTQ), filter pruning, and quantization-aware training (QAT) supporting OpenVINO and PyTorch.
- **(C)** A graphical design tool for neural network diagrams.
- **(D)** A network cable tester.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Intel NNCF provides state-of-the-art compression algorithms for deep learning models. It supports 8-bit quantization, 4-bit weight-only quantization (AWQ, GPTQ recipes), structured sparsity, and accuracy-aware tuning with minimal accuracy loss.

Question 08: D5-Q08

In Parameter-Efficient Fine-Tuning (PEFT) using LoRA (Low-Rank Adaptation), how are model weight updates represented mathematically?

- **(A)** $W = W_0 + B \cdot A$, where W_0 is frozen, and $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ are low-rank trainable matrices with rank $r \ll \min(d, k)$.
- **(B)** $W = W_0 / 2$
- **(C)** $W = W_0 \times 0$
- **(D)** $W = A + B + C + D$

> **Correct Answer**: **(A)**
> **Technical Explanation**: LoRA freezes the pre-trained weight matrix W_0 and decomposes the weight update ΔW into two low-rank matrices: B and A. For $d = 4096$ and rank $r = 16$, parameters drop from $4096 \times 4096 \approx 16.7\text{M}$ to $2 \times 4096 \times 16 \approx 131\text{K}$ (a 99.2% reduction in trainable parameters).

Question 09: D5-Q09

What is QLoRA, and how does it advance LoRA fine-tuning for resource-constrained HPC nodes?

- **(A)** It fine-tunes LLMs using quantum computing qubits.
- **(B)** It quantizes the base pre-trained model to 4-bit NormalFloat (NF4) precision, adds double quantization, and uses paged optimizers, allowing fine-tuning of 65B parameter models on modest memory allocations.
- **(C)** It removes all attention layers from the model.
- **(D)** It replaces Python code with QBasic.

> **Correct Answer**: **(B)**
> **Technical Explanation**: QLoRA (Dettmers et al., 2023) quantizes frozen weights to an information-theoretically optimal 4-bit NormalFloat (NF4) format, backpropagates gradients through the 4-bit weights into 16-bit LoRA adapter matrices, and uses paged optimizers to prevent memory spikes.

Question 10: D5-Q10

What is the primary memory bandwidth of Habana Gaudi 2 accelerators, and how does it compare to standard DDR5 server memory?

- **(A)** 2.45 TB/s using on-chip HBM2e memory, approximately 8x to 10x faster than dual-socket DDR5 server memory channels (~250-300 GB/s).
- **(B)** 50 MB/s, slower than DDR5.
- **(C)** 100 Gbps Ethernet speed.
- **(D)** Identical to DDR5.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Habana Gaudi 2 features 96 GB of HBM2e delivering 2.45 TB/s of aggregate memory bandwidth (Gaudi 3 expands to 128 GB HBM2e at 3.7 TB/s). This massive memory bandwidth makes it exceptionally fast for LLM training and token generation compared to host DDR5 memory.

Question 11: D5-Q11

What is FlashAttention, and why is it a milestone optimization for transformer architectures on HPC accelerators?

- **(A)** An Adobe Flash animation player for browser AI.
- **(B)** An exact attention algorithm that reorders computation using tiling to compute softmax without writing the intermediate $N \times N$ attention matrix to high-latency HBM, reducing memory reads/writes by orders of magnitude.
- **(C)** An algorithm that skips calculating attention weights.
- **(D)** A camera flash controller.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Standard self-attention materializes an $N \times N$ attention matrix in memory, which requires $O(N^2)$ memory reads and writes. FlashAttention uses online softmax tiling in fast SRAM / L1 cache to compute exact attention without writing the $N \times N$ intermediate matrix to HBM, dramatically speeding up long-context processing.

Question 12: D5-Q12

What is the primary metric used to evaluate LLM inference latency from the perspective of an interactive end-user?

- **(A)** Time-To-First-Token (TTFT) and Inter-Token Latency (ITL / Tokens per second).
- **(B)** Hard drive rotation speed.
- **(C)** Monitor refresh rate in Hertz.
- **(D)** File download speed.

> **Correct Answer**: **(A)**
> **Technical Explanation**: TTFT measures responsiveness (the time from submitting the prompt to the appearance of the very first token, dominated by the prefill phase). ITL measures generation fluency (the time between consecutive tokens during the decode phase).

Question 13: D5-Q13

In DeepSpeed distributed training, what is the ZeRO (Zero Redundancy Optimizer) memory optimization strategy?

- **(A)** ZeRO partitions optimizer states (Stage 1), gradients (Stage 2), and model parameters (Stage 3) across data-parallel processes, eliminating redundant memory replication without incurring model-parallel complexity.
- **(B)** ZeRO sets all model weights to zero to save RAM.
- **(C)** ZeRO disables backpropagation.
- **(D)** ZeRO runs training on zero computers.

> **Correct Answer**: **(A)**
> **Technical Explanation**: In standard Data Parallelism, every worker holds a duplicate copy of optimizer states, gradients, and model parameters. DeepSpeed ZeRO partitions these memory components across the data-parallel ranks, reducing per-device memory consumption linearly with rank count.

Question 14: D5-Q14

What library provides native integration for training and fine-tuning Hugging Face transformer models on Habana Gaudi accelerators?

- **(A)** optimum-habana
- **(B)** habana-driver-gui
- **(C)** keras-mobile
- **(D)** openvino-phone

> **Correct Answer**: **(A)**
> **Technical Explanation**: `optimum-habana` connects Hugging Face libraries (`transformers` , `diffusers` , `peft`) with Intel Habana SynapseAI software stack, providing Gaudi-optimized PyTorch execution with minimal code changes.

Question 15: D5-Q15

In OpenVINO runtime, what is the role of the `core.compile_model(model, 'CPU')` method?

- **(A)** It converts the model into a PDF document.

- **(B)** It compiles the intermediate network graph into an executable device-specific binary pipeline optimized for the host CPU's instruction set (AVX-512, AMX, thread scheduling).
- **(C)** It uploads the model to an online repository.
- **(D)** It deletes the model from RAM.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `compile_model` translates OpenVINO's device-agnostic OpenVINO IR graph into an executable pipeline for the selected hardware target ('CPU', 'GPU', 'NPU'), allocating execution memory pools and optimizing kernel scheduling.

Question 16: D5-Q16

What is Grouped-Query Attention (GQA), used in models like Llama 3 and Mistral, and how does it optimize HPC memory consumption?

- **(A)** It shares a single Key and Value head across multiple Query heads, drastically reducing the memory footprint of the KV-cache during inference while retaining multi-head attention expressive capacity.
- **(B)** It queries Google Search during inference.
- **(C)** It removes query vectors from attention.
- **(D)** It runs queries in alphabetical order.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Multi-Head Attention (MHA) maintains independent Key and Value heads for every Query head, resulting in massive KV-cache memory usage. GQA groups query heads to share a smaller number of Key/Value heads (e.g. 8 KV heads for 32 Query heads), reducing KV-cache size by 4x and speeding up decode throughput.

Question 17: D5-Q17

What is the difference between Symmetric and Asymmetric quantization in neural network compression?

- **(A)** Symmetric quantization constrains the zero-point to exactly 0 (scale only), which simplifies hardware integer multiplication units; Asymmetric quantization includes a non-zero offset (scale and zero-point), which better fits skewed non-negative activation distributions.
- **(B)** Symmetric is for text; Asymmetric is for audio.
- **(C)** Symmetric requires GPUs; Asymmetric requires CPUs.
- **(D)** There is no difference.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Symmetric quantization maps floating point range $[-M, M]$ symmetrically to $[-128, 127]$ with zero-point $Z = 0$. Asymmetric quantization maps $[\min, \max]$ to $[0, 255]$ with an explicit zero-point offset Z, which improves precision for skewed activation distributions (like ReLU/GELU) at the cost of slight hardware overhead.

Question 18: D5-Q18

In deploying high-concurrency LLM inference services, what is PagedAttention (as implemented in vLLM)?

- **(A)** An attention mechanism inspired by OS virtual memory paging that allocates KV-cache memory in non-contiguous physical memory blocks, eliminating internal and external memory fragmentation.
- **(B)** A mechanism that prints LLM responses on physical paper pages.
- **(C)** A web browser paging extension.
- **(D)** A tool to paginate HTML documents.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Standard LLM serving pre-allocates contiguous memory for maximum possible sequence lengths (e.g. 2048 tokens), resulting in 60-80% memory waste from fragmentation. PagedAttention partitions the KV-cache into fixed-size pages managed via a page table, allowing near-100% memory utilization and higher request batching concurrency.

Question 19: D5-Q19

What is the role of Intel AMX (Advanced Matrix Extensions) in accelerating LLM fine-tuning on 4th Gen Intel Xeon Scalable processors?

- **(A)** It accelerates Bfloat16 forward and backward matrix multiplications directly in core tile registers, enabling multi-node CPU clusters to fine-tune 7B/13B parameter models without discrete GPUs.
- **(B)** It compresses text datasets.
- **(C)** It controls cooling fan speeds.
- **(D)** It acts as a graphical user interface.

> **Correct Answer**: **(A)**
> **Technical Explanation**: With AMX hardware tile engines, Sapphire Rapids Xeon cores deliver up to 2 TFLOPs of BF16 compute per core. Multi-socket cluster nodes can perform distributed LoRA fine-tuning of 7B parameter models using existing enterprise CPU servers without requiring scarce discrete GPUs.

Question 20: D5-Q20

How does KV-cache quantization (e.g. quantizing KV-cache from FP16 to INT8 or INT4) enhance LLM serving capacity?

- **(A)** It halves or quarters the memory required per token in the KV-cache, allowing the server to handle 2x to 4x larger concurrent batch sizes and much longer context windows within the same DRAM capacity.
- **(B)** It makes the model output 100% accurate.
- **(C)** It deletes past conversation history.
- **(D)** It removes the need for RAM entirely.

> **Correct Answer**: **(A)**
> **Technical Explanation**: As context lengths expand to 32k or 128k tokens, the KV-cache memory easily dwarfs the base model weights. Quantizing the KV-cache to INT8/INT4 reduces per-token memory footprint by 50-75%, expanding the concurrent request capacity of the server.

Question 21: D5-Q21

When serving an OpenVINO-optimized LLM via a REST API, why is FastAPI with an asynchronous event loop (`async def`) preferred over synchronous Flask?

- **(A)** FastAPI is written in C++.
- **(B)** FastAPI handles non-blocking asynchronous I/O and streaming token generation over Server-Sent Events (SSE) concurrently across thousands of client requests without spawning a blocking OS thread per connection.
- **(C)** Flask cannot return JSON data.
- **(D)** FastAPI requires no CPU resources.

> **Correct Answer**: **(B)**
> **Technical Explanation**: FastAPI built on Starlette and `uvicorn` uses Python's asynchronous event loop (`asyncio`). It can stream generated tokens back to HTTP clients as they are produced (via `StreamingResponse` and SSE) across thousands of concurrent connections with minimal memory overhead.

Question 22: D5-Q22

In LLM token generation, what does the 'temperature' sampling parameter control?

- **(A)** The physical Celsius temperature of the CPU heatsink.
- **(B)** It scales the raw logit scores before the softmax operation ($logits/T$), where lower temperature ($T \to 0$) makes the probability distribution sharper (greedy, deterministic) and higher temperature ($T > 1$) flattens it (more diverse, creative outputs).
- **(C)** The ambient temperature in the server room.
- **(D)** The speed of the computer cooling fan.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Temperature T adjusts the entropy of the probability distribution: $P(y_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$. As $T \to 0$, it approaches an argmax greedy choice. At higher values of T, lower-probability tokens have an increased chance of selection.

Question 23: D5-Q23

What is the primary difference between Top-k sampling and Top-p (Nucleus) sampling in LLM generation?

- **(A)** Top-k restricts sampling to a fixed number k of highest-probability tokens; Top-p dynamically samples from the smallest set of tokens whose cumulative probability exceeds threshold p.
- **(B)** Top-k is for English; Top-p is for Python code.
- **(C)** Top-k uses sorting; Top-p uses hashing.
- **(D)** There is no difference.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Top-k sets a fixed pool size k regardless of the probability mass. Nucleus (Top-p) sampling dynamically expands or contracts the candidate pool based on model confidence: when confidence is high, the pool contains only 1 or 2 tokens; when confidence is low, the pool widens automatically.

Question 24: D5-Q24

What open format is used by OpenVINO to store model topologies and weights in its Intermediate Representation (IR)?

- **(A)** A `.xml` file describing the graph architecture and execution topology, paired with a `.bin` file storing binary tensor weights.
- **(B)** A `.zip` archive containing Python files.
- **(C)** An `.mp4` video file.
- **(D)** A `.csv` spreadsheet.

> **Correct Answer**: **(A)**
> **Technical Explanation**: OpenVINO IR separates graph topology from binary weights: the `.xml` file defines the neural network architecture, layer connectivity, and tensor shapes in XML format, while the `.bin` file stores the raw binary weights and biases.

Question 25: D5-Q25

In a capstone B.Tech project presentation, what key architectural advantage should students emphasize when choosing an Intel oneAPI / OpenVINO deployment over a proprietary CUDA stack?

- **(A)** Open cross-architecture portability across multi-vendor CPUs, integrated GPUs, discrete GPUs, and AI accelerators, avoiding expensive vendor lock-in while leveraging ubiquitous enterprise server hardware.
- **(B)** CUDA does not support Python.
- **(C)** Intel oneAPI makes computers immune to hardware failures.
- **(D)** OpenVINO requires zero RAM.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Proprietary stacks like CUDA lock enterprise software to a single hardware vendor. Intel oneAPI and OpenVINO adhere to open industry standards (SYCL, Level Zero), allowing models to compile and execute across diverse multi-vendor hardware (Intel CPUs, Intel GPUs, AMD chips, ARM, and accelerators), protecting software investments and repurposing existing server infrastructure.
